



Excellent question 👉 — understanding `useEffect` deeply is one of the biggest steps toward mastering React hooks. Let's break it down **step-by-step using your timer example**, and we'll explore:

1. What happens **without** `useEffect`
2. What happens **with** `useEffect` (no dependency array)
3. What happens **with** `useEffect` and an empty dependency array `[]`
4. What happens **with** `useEffect` and `[count]` in dependency array
5. Why the cleanup (`return () => clearTimeout(timer)`) matters

□ First, the Example

```
import { useState, useEffect } from "react";

function Timer() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const timer = setTimeout(() => {
      setCount((count) => count + 1);
    }, 1000);

    return () => clearTimeout(timer);
  }, [count]);

  return <h1>I've rendered {count} times!</h1>;
}

export default Timer;
```



Concept: What is `useEffect`?

`useEffect()` is a React Hook used to perform **side effects** in a component.

A **side effect** is any operation that affects something **outside** of the component rendering — examples:

- Fetching data from an API
- Setting a timer (`setTimeout`, `setInterval`)
- Accessing browser APIs (e.g., `localStorage`)
- Manually updating the DOM
- Setting up event listeners

React's rendering is **pure** — meaning it should only compute what the UI looks like, not *do stuff*.

`useEffect()` lets you *do stuff* **after** React has updated the screen.

❑ 1. Without `useEffect`

If you wrote:

```
function Timer() {
  const [count, setCount] = useState(0);

  const timer = setTimeout(() => {
    setCount(count + 1);
  }, 1000);

  return <h1>I've rendered {count} times!</h1>;
}
```

🔍 What happens:

- On the **first render**, React runs this component function.
- It creates a timer to update the count after 1 second.
- When that timer fires, `setCount` changes state → React re-renders.



- **During re-render**, this line runs *again* → creates another timer.
- That means **every render creates a new timer** → quickly becomes **infinite loop** of timers!
- Soon, hundreds of timers are running → browser slows down or crashes.

✗ Problem:

You're performing a *side effect* directly in the render function — something React **forbids**. It breaks the rendering model because React cannot control those effects.

❑ 2. Using `useEffect` (no dependency array)

```
useEffect(() => {  
  const timer = setTimeout(() => {  
    setCount(count + 1);  
  }, 1000);  
});
```

🔍 What happens:

- React renders your component.
- After each render, React runs `useEffect`'s function.
- Since there's **no dependency array**, it runs **after every render**.
- Each render sets a new timeout → again multiple timers → still infinite loop.

So even with `useEffect`, **not adding dependencies** can cause repeated execution and bad behavior.

❑ 3. Using `useEffect` with an empty dependency array `[]`

```
useEffect(() => {  
  const timer = setTimeout(() => {  
    setCount((count) => count + 1);  
  }, 1000);  
}, []);
```



```
}, []);
```

🔍 What happens:

- The empty array `[]` tells React:
📖 “Run this effect **only once** — when the component mounts.”

🔄 Flow:

1. Component renders first time.
2. Effect runs once — sets a timeout to increase count after 1 second.
3. After 1 second → count becomes 1 → component re-renders.
4. But since dependency array is `[]`, the effect **doesn't run again**.

📋 Result:

- The counter updates **only once**, then stops.
- So you'll see:
- I've rendered 0 times!
→ after 1 sec:
I've rendered 1 times!
→ and then it stops.

❏ 4. Using `useEffect` with `[count]` dependency

```
useEffect(() => {  
  const timer = setTimeout(() => {  
    setCount((count) => count + 1);  
  }, 1000);  
  
  return () => clearTimeout(timer);  
}, [count]);
```



🔍 What happens:

- The effect runs every time `count` changes.
- When the timer fires and updates `count`, React re-renders → effect runs again.
- Each time:
 - It creates a new timeout for 1 second.
 - Cleans up the previous timeout (important to avoid overlap).
- This loop continues forever → gives a **1-second counter**.

🔄 Flow:

Step Count Value

What Happens

1	0	<code>useEffect</code> sets timer → after 1s: <code>count = 1</code>
2	1	React re-renders → new timer set → after 1s: <code>count = 2</code>
3	2	React re-renders → new timer set → ... continues

✓ Output:

```
I've rendered 0 times!  
I've rendered 1 times!  
I've rendered 2 times!  
I've rendered 3 times!  
...
```

❏ **Cleanup Function:** `return () => clearTimeout(timer)`

React calls this cleanup function:

- Before the component is removed (unmounted).
- Before running the effect again (when dependencies change).

This ensures:

- No overlapping timers.
- No memory leaks.



- No outdated timeouts running in the background.

So each time the count updates:

- React clears the old timeout.
- Sets a new one → clean and predictable.

Summary Table

Scenario	Effect Runs When	Result	Problem
✗ No <code>useEffect</code>	Every render (in render body)	Infinite timers	Crashes
⚠ <code>useEffect()</code> no deps	After every render	Infinite timers	Bad
⚠ <code>useEffect([])</code>	Only once (on mount)	Updates once	Stops after 1 increment
✓ <code>useEffect([count])</code>	After each count change	Updates every second	Perfect timer

Key Takeaways

- `useEffect` = perform side effects after render.
- `[]` = run only once.
- `[someValue]` = re-run when that value changes.
- No array = re-run after every render.
- Always clean up (timers, intervals, event listeners).